

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_6e519d4b-9e6\agent-a128a22a62b05396b.jsonl`  
- SHA-256: `5c3f1701832639c2097dfc5b24dcfe46d6e2e42aa63e78b5e69994d5776785b7`  
- Source modified: `2026-06-22T14:45:24+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_6e519d4b-9e6`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T14:44:22.148Z)

Read in C:/Users/avidu/Projects/bhi-cloudrun: backend/config/presets.py (the cloud-serving preset), backend/config/models.py (DeploymentConfig / compute_target / storage), backend/config/loader.py (how a profile applies), backend/storage/__init__.py + backend/storage/gcs.py (how gs:// read/write + acquire_local_input work). Map: how compute_target=cloud_run + storage=gcs get set + flow to get_compute_backend; and the ON-BUCKET OUTPUT FORMAT a cloud `worker infer` job writes to --out-uri (the exact files + their schema ? intervals.csv columns, report.json keys, the success/done marker) that the API would need to INGEST back into its DB. Quote file:line.

2. assistant (2026-06-22T14:44:24.990Z)

I'll map the cloud-serving config flow and the on-bucket output format. Let me read the requested files.

3. user (2026-06-22T14:44:28.237Z)

You are not currently on a branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

```
git pull <remote> <branch>
```

4. user (2026-06-22T14:44:28.267Z)

```
1      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9      built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (`config.json` `deployment.profile` or the `DEPLOYMENT_PROFILE` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (`profile=""` ==
15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
```

```

18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55             "storage": {"backend": "gcs"},
56         },
57         "cpu-mock": {
58             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61             "storage": {"backend": "local"},
62         },
63     }
64
65
66     def is_known_profile(profile: str) -> bool:
67         """True for a non-empty, recognised profile name."""
68         return bool(profile) and profile in PROFILE_PRESETS
69
70
71     def preset_for(profile: str) -> dict[str, Any]:
72         """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the

```

```

73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""
106     return _cuda_available()
107
108
109 def suggest_profile(
110     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111 ) -> str:
112     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114     Signals, in priority order:
115
116     - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117     - ``CI`` truthy                           ? ``"cpu-mock"``
118     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119     - otherwise                               ? ``""`` (no strong suggestion)
120
121     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124
125     """
126     e = os.environ if env is None else env
    if e.get("K_SERVICE"):

```

```

127         return "cloud-serving"
128     if _truthy(e.get("CI")):
129         return "cpu-mock"
130     if probe_cuda and _cuda_available():
131         return "truly-local"
132     return ""
133

```

5. user (2026-06-22T14:44:28.318Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
33     it honours
34     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
35     relative) when
36     # the env is unset, exactly the pre-P0a value. (The old module-level
37     ``DEFAULT_CONFIG_PATH`` constant
38     # was dropped: it had no external consumers and a frozen-at-import snapshot would
39     silently diverge
40     # from a later RALLY_APP_HOME.)
41
42     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
43         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
44         scalars
45         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
46         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
47         profile
48         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
49         out = dict(base)
50         for key, val in override.items():
51             if isinstance(val, dict) and isinstance(out.get(key), dict):
52                 out[key] = _deep_merge(out[key], val)
53             else:
54                 out[key] = val

```

```

50         return out
51
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ```` if
a bare
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
choice)
72         dep = file_data.get("deployment")
73         if isinstance(dep, dict):
74             prof = dep.get("profile")
75             if isinstance(prof, str) and prof.strip():
76                 return prof.strip()
77         return ""
78
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
81         """Dotted paths in `data` that the typed config will not honor.
82
83         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
84         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
85         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
86         user's intent is lost ? collect every such key so load_config can warn.
87         """
88         unknown: list[str] = []
89         fields = model_cls.model_fields
90         for key, val in data.items():
91             if key not in fields:
92                 unknown.append(prefix + key)
93                 continue
94             ann = fields[key].annotation
95             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
96                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
97         return unknown
98
99
100     def _get_builtin_defaults() -> dict[str, Any]:
101         """Return a plain dict of the current built-in defaults.
102
103         The AppConfig model is the single source of truth for defaults.
104         """

```

```

105         return AppConfig().model_dump(mode="json")
106
107
108     def load_config(path: str | Path | None = None) -> AppConfig:
109         """Load configuration with the following precedence:
110
111         1. Built-in defaults defined in the Pydantic models.
112         2. Values from the JSON file (if present and readable).
113         3. (Future) environment / CLI overrides.
114
115         Missing file or unreadable file ? returns a fully defaulted AppConfig.
116         Unknown keys in the file are tolerated (extra="allow" on the model).
117         """
118         cfg_path = Path(path) if path is not None else default_config_path()
119
120         file_data: dict[str, Any] = {}
121         if cfg_path.exists():
122             try:
123                 with open(cfg_path, "r", encoding="utf-8") as f:
124                     file_data = json.load(f)
125             except Exception as exc:
126                 # Non-fatal: continue with defaults + whatever we could parse.
127                 # In production we might log here.
128                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
129
130             # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
131             # `null`) would
132             # otherwise crash startup: a truthy non-mapping hits `.items()` in
133             # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
134             # loader's "bad input is non-fatal" contract.
135             if not isinstance(file_data, dict):
136                 logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
137                               cfg_path, type(file_data).__name__)
138             file_data = {}
139
140         if file_data:
141             unknown = _unknown_key_paths(file_data, AppConfig)
142             if unknown:
143                 logger.warning(
144                     "[CONFIG WARNING] %s contains keys the typed config does not "
145                     "recognize (typo'd or removed knobs ? top-level extras are kept "
146                     "but unread; unknown section keys are silently dropped): %s",
147                     cfg_path, ", ".join(sorted(unknown)),
148                 )
149
150         # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
151         # at
152         # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
153         # ONLY when
154         # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
155         # With no
156         # profile the merge is byte-identical to the pre-M1 loader, so the default path is
157         # unchanged.
158         defaults = _get_builtin_defaults()
159         profile = _resolve_explicit_profile(file_data)
160
161         if profile and not is_known_profile(profile):
162             # Only an unrecognised value from config.json reaches here (env typos already
163             # fell back
164             # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
165             the

```

```

159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,
169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180             elif ct and canonical and ct != canonical:
181                 logger.warning(
182                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
183                     "config.json overrides it to %r ? honoring the explicit override.",
184                     profile, canonical, ct,
185                 )
186                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                             profile, merged["deployment"].get("compute_target"))
188     else:
189         # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
190         merged = {**defaults, **file_data}
191         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192         if suggestion:
193             logger.debug(
194                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
195                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
196                 "auto-applies ? D15).", suggestion,
197             )
198
199     return AppConfig.model_validate(merged)
200
201
202 def save_default_config(path: str | Path | None = None) -> Path:
203     """Write a fresh default config.json.
204
205     Used by scripts/doctor.py diagnostics and the --setup CLI path.
206     Overwrites if the file exists.
207     """
208     cfg_path = Path(path) if path is not None else default_config_path()
209     cfg = AppConfig()
210     cfg_path.parent.mkdir(parents=True, exist_ok=True)
211     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212     return cfg_path

```

```

213
214
215     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
216     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
217     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
218     # user gets ? a torch-free, offline, zero-setup experience:
219     # * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
220     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
bills on
221     # Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
222     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
223     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
224     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
225         "sport": "badminton",
226         "deployment": {"profile": "truly-local"},
227         "indexing": {"default_segmenter": "motion", "use_gpu": False},
228     }
229
230
231     def write_light_default_config(
232         path: str | Path | None = None, *, overwrite: bool = False
233     ) -> tuple[Path, bool]:
234         """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
235
236         Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
237         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
238         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
239         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
240         """
241         cfg_path = Path(path) if path is not None else default_config_path()
242         if cfg_path.exists() and not overwrite:
243             return cfg_path, False
244         cfg_path.parent.mkdir(parents=True, exist_ok=True)
245         cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n",
encoding="utf-8")
246         return cfg_path, True
247
248
249     # Backwards-compatible alias used by some older call sites during transition.
250     def get_default_config() -> dict[str, Any]:
251         """Return defaults as a plain dict (for transitional use only)."""
252         return _get_builtin_defaults()
253

```

6. user (2026-06-22T14:44:28.780Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10

```



```

11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
66         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
67         sport: str = "badminton"
68         wsl_stage_dir: str = "~/clips"
69         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
70         velocity_threshold: float = 0.02
71         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
72         ---
73
74         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
75         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4

```

```

CPU fallback
73         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74         device: str = "cuda"          # auto | cuda | mps | cpu
75         python_bin: str = "python"    # the `wasb` conda env python (invoked by path, no
`conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79         keep_frames: bool = False
80         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81         min_free_gb: float = 0.0
82         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                               behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.

```

```

118     min_players: int = Field(default=0, ge=0)
119     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120     court_polygon: Optional[list[list[float]]] = None
121     # Net line for the person two-sided-motion split (person features only ? the shuttle
122     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123     net_axis: Literal["x", "y"] = "y"
124     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
135         pipeline/detectors/{stub,native_wasb}_runner.py.
136         detector_impl: str = "auto"
137         motion_threshold: float = 0.08
138         min_segment_duration: float = 3.0
139         dead_time_merge_gap: float = 2.0
140         chunk_duration: float = 90.0
141         chunk_overlap: float = 10.0
142         detector_model: str = "fasterrcnn_resnet50_fpn"
143         detector_score_threshold: float = 0.5
144         yolo_velocity_threshold: float = 0.15
145         yolo_frame_skip: int = 3
146         yolo_tracker: str = "bytetrack.yaml"
147         yolo_prefetch_workers: int = 2
148         min_action_window_duration: float = 2.0
149         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
150         # = a window
151         # longer than this many seconds is recursively split at its largest internal
152         # inactivity gap (?
153         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
154         # swallow
155         # several rallies + the dead time between them. Never merges, only cuts (recall
156         # can't drop).
157         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
158         # Measured n=13
159         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
160         # p=0.0005; cap=6
161         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
162         # 0.651?0.161). ***
163         max_window_duration: float = 6.0
164         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
165         point
166         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
167         # inactivity gap
168         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
169         # is
170         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
171         # bound.
172         # Only cuts (recall can't drop). OFF by default until measured/promoted.
173         window_hard_cap: bool = False
174         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
175         # slowly
176         # (picked up / walked) above velocity_thresh, so the window over-extends its END
177         # (the measured
178         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
179         # trailing
180         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
181         # cuts.
182         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but

```

```

significant
167      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169      window_inactivity_end: float = 1.5
170      inactivity_rally_thresh: float = 0.20
171      inactivity_min_density: float = 0.30
172      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180      window_blob_fallback: bool = False
181      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185      window_blob_factor: float = 4.0
186      log_yolo_candidates: bool = True
187      store_yolo_candidates: bool = True
188      ai_handoff_padding: float = 2.0
189      ai_max_workers: int = 5
190      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191      # chunks of a high-bitrate (4K) source blow past the provider upload cap
192      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193      # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194      # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195      # Matches gemini_refine's --clip-scale-width default.
196      ai_chunk_scale_width: int = 1280
197      # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198      # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199      # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200      # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201      # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202      skip_ai_handoff: bool = False
203      # Optional debug knob: trim every video to the first N seconds before processing.
204      debug_duration: Optional[float] = None
205
206      tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207      wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208      fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210      @field_validator("default_segmenter")
211      @classmethod
212      def segmenter_must_be_registered(cls, v: str) -> str:
213          # Registry check happens at runtime in main/segmenter selection.

```

```

214         # Here we just allow any string for forward compatibility.
215         return v
216
217     @model_validator(mode="after")
218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
220         # step makes `while t < duration: t += step` loop forever, growing memory before
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
249         # fraction of frame height
250         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
251         box: bool = True # faint backing box behind the text for legibility
252         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
253         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
254         right"
255
256     class StitchingConfig(BaseModel):
257         begin_padding: float = 0.5
258         end_padding: float = 0.5
259         use_cache: bool = False
260         min_shot_count: int = 1
261         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
262         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
263         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
264         # the
265         # fully-local no-AI path, whereas duration is derived from the rally's own
266         # start/end. The
267         # local detector over-fires and its false positives are mostly SHORT (motion blips,
268         # background-court fragments), so this keeps the reel to the eye-catching long
269         # rallies.
270         # OFF by default ? the detector output is unchanged, only which rallies reach the
271         # reel.
272         min_rally_duration: float = Field(default=0.0, ge=0.0)

```

```

268     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run endpoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = "" # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-

```

```

breaking).
320     output_root: str = ""
321     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324     # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326     # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329     input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330     input_root: str = ""
331     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335     # the legacy flat layout is used unchanged.
336     single_folder_per_video: bool = False
337     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338     workspace_root: str = ""
339     # Cloud namespace prefix under the root so per-video folders sit tidily beside
340     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>). "" = root
directly.
341     workspace_prefix: str = "workspaces"
342     local: dict[str, Any] = Field(default_factory=dict)
343     s3: dict[str, Any] = Field(default_factory=dict)
344     gcs: dict[str, Any] = Field(default_factory=dict)
345     gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""

```

```

366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
372         NULL = today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
373         the versioned ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
374         pricebook version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
375         owner inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
378         follow-up wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
379         ledger + pricebook + the recording seam; usage emission (the metering hooks) is a separate
380         step."""
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to
383         a real one
384         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore
385         market)
386         margin: float = 0.0 # resale markup (?0); 0 = bill at
387         cost
388
389     class AppConfig(BaseModel):
390         """Root configuration object.
391
392         All runtime configuration should be accessed via an instance of this class
393         (or the loader) rather than raw dict lookups.
394
395         """
396         sport: Sport = "badminton"
397         ai_provider: str = "gemini"
398         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
399         flagship;
400         # the alias never deprecates underneath us. Eval reports record the run date
401         alongside.
402         ai_model: str = "gemini-flash-latest"
403
404         validation: ValidationConfig = Field(default_factory=ValidationConfig)
405         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
406         output: OutputConfig = Field(default_factory=OutputConfig)
407         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
408         security: SecurityConfig = Field(default_factory=SecurityConfig)
409         logging: LoggingConfig = Field(default_factory=LoggingConfig)
410         storage: StorageConfig = Field(default_factory=StorageConfig)
411         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
412         billing: BillingConfig = Field(default_factory=BillingConfig)
413
414         # Allow extra keys during transition so existing config.json files do not break.
415         model_config = {
416             "extra": "allow",
417             "validate_assignment": True,
418         }
419
420         # Temporary migration shim for legacy dict-style access (config.get("section",
421         {}).get(...)).
422         # Will be removed once all call sites (validators, segmenters, etc.) move to typed

```



```

418         # attribute access / the typed config. A nested *section* is returned as a plain
dict
419         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
420         # section model instead breaks it (a model has no .get), which bit every validator
on
421         # real runs. Scalar leaves are returned as-is.
422         def get(self, key: str, default: Any = None) -> Any:
423             _MISSING = object()
424             val: Any = _MISSING
425             if key in type(self).model_fields or hasattr(self, key):
426                 val = getattr(self, key, _MISSING)
427             if val is _MISSING:
428                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
429                 # discovered dynamically from the model fields, so a future top-level
section
430                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
431                 for fname in type(self).model_fields:
432                     section = getattr(self, fname, None)
433                     if isinstance(section, BaseModel) and hasattr(section, key):
434                         val = getattr(section, key)
435                         break
436             if val is _MISSING:
437                 return default
438             if isinstance(val, BaseModel):
439                 return val.model_dump(mode="json")
440             return val
441
442         def __getitem__(self, key: str) -> Any:
443             val = self.get(key)
444             if val is None and key not in type(self).model_fields:
445                 # Re-raise like a real dict for unknown top-level
446                 raise KeyError(key)
447             return val
448
449
450         # Convenience type alias for callers that want to annotate
451         Config = AppConfig
452

```

7. user (2026-06-22T14:44:28.877Z)

```

1         """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3         URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4         lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5         ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6         today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7         Google Drive (Google Drive for Desktop) as plain files.
8
9         Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10        ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11        passthrough. See ``docs/DECOUPLED_COMPUTE``.
12        """
13        from __future__ import annotations
14
15        import os
16        import tempfile
17        from typing import Any, Dict, Iterator, Optional
18

```

```

19 from backend.storage.base import StorageBackend
20 from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22 __all__ = [
23     "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24     "get_storage", "fetch", "put", "list_uris",
25     "acquire_local_input", "input_is_local", "resolve_input_ref",
26 ]
27
28
29 def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30     """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31     if storage_config is None:
32         return {}
33     if hasattr(storage_config, "model_dump"):
34         return storage_config.model_dump() # type: ignore[no-any-return]
35     return dict(storage_config)
36
37
38 def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39     """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40     config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41     callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44     ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45     sc = _storage_dict(storage_config)
46     uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                             input_root=sc.get("input_root", ""))
48     return parse_uri(uri)
49
50
51 def input_is_local(raw: str, storage_config: Any = None) -> bool:
52     """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53     plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54     that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55     ``resolve_input_ref``)."""
56     return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59 def acquire_local_input(raw: str, storage_config: Any = None, *,
60                         scratch_parent: "str | None" = None) -> str:
61     """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63     Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64     byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65     non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66     path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67     regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68     worker's ``cmd_infer`` already ships."""
69     sc = _storage_dict(storage_config)
70     ref = resolve_input_ref(raw, sc)

```

```

71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):

```

```
126         yield r.uri
127
```

8. user (2026-06-22T14:44:29.164Z)

```
1      """Google Cloud Storage backend ? first-class object buckets.
2
3      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4      install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6      never from config. A ``client`` may be injected for tests.
7      """
8      from __future__ import annotations
9
10     from datetime import timedelta
11     from typing import Any, Iterator, Optional
12
13     from backend.storage.base import StorageBackend
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class GCSStorage(StorageBackend):
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                     signed_url_ttl: int = 3600, **_ignored) -> None:
20             self._ttl = signed_url_ttl
21             if client is not None:
22                 self._client = client
23             return
24         from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25         self._client = storage.Client(project=project)
26
27         def _blob(self, ref: StorageRef):
28             return self._client.bucket(ref.bucket).blob(ref.key)
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             if dst is None:
32                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33             import os
34             dst = str(dst)
35             if os.path.exists(dst) and not overwrite:
36                 return dst
37             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38             self._blob(ref).download_to_filename(dst)
39             return dst
40
41         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43             return ref
44
45         def exists(self, ref) -> bool:
46             return bool(self._blob(ref).exists())
47
48         def stat(self, ref) -> StorageStat:
49             b = self._blob(ref)
50             b.reload()
51             return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                               content_type=b.content_type)
53
54         def list(self, prefix) -> Iterator[StorageRef]:
55             for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                 yield StorageRef("gcs", prefix.bucket, blob.name,
```

```
f"gcs://{prefix.bucket}/{blob.name}")
57
58     def delete(self, ref) -> None:
59         self._blob(ref).delete()
60
61     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62         return self._blob(ref).generate_signed_url(
63             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64
```

9. assistant (2026-06-22T14:44:32.034Z)

Now let me read the worker's `cmd\_infer` to find the exact on-bucket output format written to `--out-uri`. Let me locate it.

10. user (2026-06-22T14:44:34.270Z)

Found 3 files
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker__main__.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud_run.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py

11. user (2026-06-22T14:44:34.797Z)

Found 10 files
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker__main__.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\gt_loader.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\fusion_golden.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\annotations.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\calibrate_wasb.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud_run.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\annotations\automated_provider.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\annotations\base.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\annotations\file_provider.py

12. user (2026-06-22T14:44:37.719Z)

```
1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. **Read** the marker (it carries `video_id` + the worker's returncode) ? fetch
12        `outputs/<video_id>/report.json` ? `ComputeResult`.
13
14    The dispatched container is forced to run **INLINE** (`compute_target=local_gpu` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
```

```

18     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19     owner-verified on the M7 real run.
20     """
21
22     from __future__ import annotations
23
24     import json
25     import logging
26     import os
27     import tempfile
28     import uuid
29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34     from backend.storage import fetch, parse_uri
35     from backend.storage import get_storage
36
37     LOG = logging.getLogger("compute.cloud_run")
38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency)."
```

```

72         ) from exc
73         client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74         name = client.job_path(project, region, job_name)
75         overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77         )
78         op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79         return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86             project: Optional[str] = None, storage_config: Optional[dict] = None,
87             policy: ExecutionPolicy = DEFAULT_POLICY,
88             token: Optional[str] = None,
89             container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name
92             self._region = region
93             self._project = project
94             self._storage_config = storage_config or {}
95             self._policy = policy
96             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98             self._token = token
99             self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                 "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
project=self._project)
115             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
self._job_name, execution, done_uri)
116
117             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
label="cloud-run-infer", logger=LOG)
118             video_id = str(marker.get("video_id", ""))
119             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
120             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
121             rc = int(marker.get("returncode", 1))
122             if not video_id:
123                 LOG.warning("cloud-run: job=%s completion marker present but

```

```

empty/unreadable (no "
127         "video_id) ? treating as failure", self._job_name)
128         rc = rc or 1
129         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134     # --- storage-backed completion signal (bucket-poll) ---
135     def _read_marker(self, done_uri: str) -> Optional[dict]:
136         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

13. user (2026-06-22T14:44:37.874Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16     from __future__ import annotations
17
18     import argparse
19     import csv

```



```

20     import json
21     import logging
22     import os
23     import re
24     import sys
25     import tempfile
26     from typing import Any, List
27
28     from backend import storage
29     from backend.compute import ComputeJobSpec, get_compute_backend
30     from backend.config import (
31         StartupCheckError,
32         enforce_startup_checks,
33         load_config,
34         probe_cuda_available,
35     )
36     from backend.infrastructure.database import Database
37     from backend.pipeline.segmenters.base import segmenter_registry
38     from backend.pipeline.validators.base import registry as validator_registry
39     from backend.reporting import emit_indexer_report
40     from backend.results import Failure
41     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
42     from backend.utils.hashing import compute_video_id
43
44     logger = logging.getLogger("backend.worker")
45
46
47     def _coerce(v: str) -> Any:
48         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
49 str."""
50         low = v.lower()
51         if low in ("true", "false"):
52             return low == "true"
53         for cast in (int, float):
54             try:
55                 return cast(v)
56             except ValueError:
57                 pass
58         return v
59
60     def _apply_overrides(config: Any, sets: List[str]) -> None:
61         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
62 indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71                 setattr(obj, parts[-1], _coerce(val))
72
73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
75 friendly
76 artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow([
82                     f'{float(s.get("start_time", 0.0)):.3f}',

```

```

82         f'{float(s.get("end_time", 0.0)):.3f}',
83         s.get("shot_count", ""),
84         s.get("ending_reason", s.get("shot_count_confidence", "")),
85     ])
86
87
88     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),
95                             "end": float(s.get("end_time", 0.0))} for s in segments],
96             }, f, indent=2)
97
98
99     def _run_startup_checks(config: Any) -> bool:
100         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
101         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
102         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
103         (returns True, ZERO work) when no deployment.profile is selected.
104
105         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
106         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
107         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
108         no-op of work, not just of output."""
109         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
110         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
111         try:
112             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
113             return True
114         except StartupCheckError:
115             return False # already logged at ERROR by enforce_startup_checks
116
117
118     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
119         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
120         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
122
123         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
124         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
125         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
126         from backend.infrastructure.execution_policy import PolicyTimeout
127
128         spec = ComputeJobSpec(
129             video_uri=args.video_uri, out_uri=args.out_uri,
130             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
131         )
132         try:

```

```

133         result = compute.run_infer(spec)
134     except PolicyTimeout as e:
135         logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
136         return 4
137     logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
138                 result.video_id, result.returncode, result.outputs_uri)
139     return result.returncode
140
141
142     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
143                           status: str, storage_cfg: dict, scratch: str) -> None:
144         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
145         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
146         try:
147             marker = {"video_id": video_id, "returncode": returncode,
148                       "segment_count": segment_count, "status": status}
149             local = os.path.join(scratch, "_done.json")
150             with open(local, "w", encoding="utf-8") as f:
151                 json.dump(marker, f)
152             storage.put(local, done_uri, config=storage_cfg)
153             logger.info("[worker] wrote completion marker -> %s", done_uri)
154         except Exception as e: # never fail a good run because the marker push hiccuped
155             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
156
157
158     def cmd_infer(args: argparse.Namespace) -> int:
159         config = load_config(args.config) if args.config else load_config()
160         _apply_overrides(config, args.set)
161         if not _run_startup_checks(config):
162             return 2
163
164         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
165         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
166         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
167         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
168         compute = get_compute_backend(config)
169         if compute is not None:
170             return _dispatch_remote(compute, args)
171
172         storage_cfg = config.storage.model_dump()
173
174         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
175         os.makedirs(scratch, exist_ok=True)
176         config.output.output_dir = scratch
177
178         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
179         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
180         print(f"[worker] pulled {args.video_uri} -> {local_video}")
181
182         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
183         video_id = compute_video_id(local_video)
184
185         # 3. VALIDATE (same registry as the main CLI).
186         val_results, val_context = validator_registry.run_all_with_context(local_video,

```

```

config)
187     failures = [r for r in val_results if not r.passed]
188     db = Database(os.path.join(scratch, config.output.db_name))
189     db.add_video(video_id, local_video, "failed" if failures else "processed",
190                 [r.model_dump() for r in val_results])
191
192     segments: List[dict] = []
193     segmenter_actual = None
194     segmentation_ran = False
195     status = "failed"
196     try:
197         if failures:
198             print("[worker] validation FAILED: "
199                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
200             return 2
201         seg_name = args.segmenter or config.indexing.default_segmenter
202         segmenter_cls = segmenter_registry.get(seg_name)
203         if not segmenter_cls:
204             print(f"[worker] unknown segmenter: {seg_name!r} "
205                   f"(have {list(segmenter_registry.list_available())})")
206             return 2
207         segmenter_actual = seg_name
208         result = segmenter_cls(db, config).process_video(video_id, local_video,
209 max_frames=args.max_frames)
210         segmentation_ran = True
211         if isinstance(result, Failure):
212             print(f"[worker] segmenter FAILED: {result.message}")
213             return 3
214         segments = result.value if hasattr(result, "value") else result
215         status = "success"
216         # Loud, never-silent diagnostic: a 0-segment success is almost always a
217         # empty match ? the cold-start failure mode (the substrate detector yielded no
218         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
219         # for more).
220         if not segments:
221             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
222                             or getattr(config.indexing, "detector_impl", None) or
223                             "auto")
224             logger.warning(
225                 "infer produced 0 segments for video_id=%s (segmenter=%s,
226                 detector_impl=%s). "
227                 "The detector substrate yielded no usable trajectories/windows ? check
228                 the detector "
229                 "logs above; common causes: native runner UNAVAILABLE
230                 (weights/repo/WASB_PYTHON env), "
231                 "the WASB env produced no detections, or the input resolution differs
232                 from the tuned "
233                 "proxy resolution. Re-run with -v for the native command + frame
234                 counts.",
235                 video_id, segmenter_actual, detector_impl,
236             )
237         finally:
238             # Best-effort per-video observability report (never raises) ? parity with the
239             # main CLI.
240             emit_indexer_report(
241                 db=db, video_id=video_id, video_path=local_video, config=config,
242                 val_results=val_results, val_context=val_context,
243                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
244                 was_reingest=False, segmentation_ran=segmentation_ran,
245             )
246
247     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
248     out_local = os.path.join(scratch, "outputs", video_id)

```

```

240     os.makedirs(out_local, exist_ok=True)
241     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
242     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
243
244     out_prefix = args.out_uri.rstrip("/")
245     pushed = []
246     for fn in sorted(os.listdir(out_local)):
247         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
248         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
249         pushed.append(uri)
250
251     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
252     for u in pushed:
253         print("    ", u)
254
255     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
256     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
257     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
258     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
259     if getattr(args, "done_uri", None):
260         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
261     return 0
262
263
264     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
265     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
266     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
267
268     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
269     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
270     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
271     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
272
273
274     def _label_clip_name(fname: str) -> str:
275         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
276
277         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
278         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
279
280         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
281         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
282         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
283         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
284
285         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
286         video stem, so every clip is skipped and train aborts with ``<2 videos``

```

```

(DATA_IN_GCS $7b #1).
287     ""
288     name = fname.replace(".rallies.csv", "").replace(".csv", "")
289     return _LABEL_DATE_PREFIX.sub("", name)
290
291
292     def _probe_video_fps_width(path: str):
293         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
294
295         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
296         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
297         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
298         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
299         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
300         rather than guesses.
301         ""
302         import json
303         import subprocess
304         try:
305             out = subprocess.check_output(
306                 [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
307                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
308                 stderr=subprocess.DEVNULL).decode()
309             st = (json.loads(out).get("streams") or [{}])[0]
310             num, _ , den = str(st.get("r_frame_rate", "0/1")).partition("/")
311             fps = float(num) / float(den or "1")
312             width = float(st.get("width") or 0)
313             if fps > 0 and width > 0:
314                 return fps, width
315         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
316             pass
317         return None
318
319
320     def _resolve_train_detector(args: argparse.Namespace, config: Any):
321         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
322         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
323         stub=CPU mock). Returns the runner, or prints an error + returns None.
324
325         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
326         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
327         no shuttle detector and is rejected.
328         ""
329         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
330
331         seg_cls = segmenter_registry.get(args.segmenter)
332         if not seg_cls:
333             print(f"[worker] unknown segmenter: {args.segmenter!r} "
334                   f"(have {list(segmenter_registry.list_available())})")
335             return None
336         seg = seg_cls(None, config)
337         if not isinstance(seg, TrajectoryHybridSegmenter):
338             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
339                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
340         return None

```

```

341     try:
342         runner, _ = seg._resolve_runner(config.get("indexing", {}))
343     except NotImplementedError as e:
344         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
345         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
346         traceback.
347         print(f"[worker] detector not available: {e}")
348         return None
349     ok, msg = runner.healthcheck()
350     if not ok:
351         print(f"[worker] detector environment not ready: {msg}")
352         return None
353     print(f"[worker] detector ready ({args.segmenter}): {msg}")
354     return runner
355
356 def cmd_train(args: argparse.Namespace) -> int:
357     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
358     shell out
359     to training.gen0.harness (backend must NOT import training) -> push the artifact
360     bundle.
361     Two trajectory sources (the train pipeline + harness are identical either way):
362     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
363     synthetic
364     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
365     CI.
366     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
367     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
368     This
369     is the M3.5 "first real training" path; only the trajectory source flips.
370     """
371     import subprocess
372
373     config = load_config(args.config) if args.config else load_config()
374     _apply_overrides(config, args.set)
375     if not _run_startup_checks(config):
376         return 2
377     storage_cfg = config.storage.model_dump()
378
379     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
380     labels_dir = os.path.join(scratch, "labels")
381     traj_dir = os.path.join(scratch, "trajectories")
382     videos_dir = os.path.join(scratch, "videos")
383     os.makedirs(labels_dir, exist_ok=True)
384     os.makedirs(traj_dir, exist_ok=True)
385
386     corpus = args.corpus_uri.rstrip("/")
387     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
388     config=storage_cfg)
389         if u.lower().endswith(".csv"))
390     if len(label_uris) < 2:
391         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
392         {len(label_uris)} "
393         f"under {corpus}/labels/")
394         return 2
395
396     # Real-detector source: resolve the runner once + index the bucket's videos/ by
397     stem.
398     runner = None
399     video_by_stem: dict = {}
400     if args.trajectory_source == "detector":
401         os.makedirs(videos_dir, exist_ok=True)
402         runner = _resolve_train_detector(args, config)
403         if runner is None:

```

```

397         return 2
398     for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
399         vname = storage.parse_uri(vuri).name
400         if not vname.lower().endswith(_VIDEO_EXTS):
401             continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
402         video_by_stem[os.path.splitext(vname)[0]] = vuri
403
404     print(f"[worker] trajectory source: {args.trajectory_source}")
405     specs: List[dict] = []
406     for uri in label_uris:
407         fname = storage.parse_uri(uri).name # e.g.
2026-06-22_GX020094.rallies.csv
408         name = _label_clip_name(fname) # -> GX020094 (date
prefix stripped)
409         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
410         if args.trajectory_source == "detector":
411             vuri = video_by_stem.get(name)
412             if not vuri:
413                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
414                 continue
415                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
416                                     config=storage_cfg)
417                 video_id = compute_video_id(local_video)
418                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
419                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
420                 meta = _probe_video_fps_width(local_video)
421                 if meta is None:
422                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
423                     continue
424                     fps, frame_width = meta
425                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
426                     if not traj:
427                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
428                         continue
429                         specs.append({"name": name, "trajectory": traj, "labels": local_label,
430                                     "fps": fps, "frame_width": frame_width})
431                     else:
432                         from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
433                         traj = os.path.join(traj_dir, f"{name}_traj.csv")
434                         synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
435                         specs.append({"name": name, "trajectory": traj, "labels": local_label,
436                                     "fps": args.fps, "frame_width": args.frame_width})
437
438     if len(specs) < 2:
439         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
440         return 2
441     manifest_path = os.path.join(scratch, "manifest.json")
442     with open(manifest_path, "w", encoding="utf-8") as f:
443         json.dump(specs, f, indent=2)
444     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
445     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
446
447     out_dir = os.path.join(scratch, "artifacts")

```



```

448     cmd = [sys.executable, "-m", "training.gen0.harness",
449             "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
450     if args.floor:
451         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
452                        config=storage_cfg)
453                        if "://" in args.floor else args.floor)
454     cmd += ["--floor", floor_local]
455     print("[worker] training:", " ".join(cmd))
456     rc = subprocess.run(cmd).returncode
457     if rc != 0:
458         print(f"[worker] gen0 harness failed (rc={rc})")
459         return rc
460
461     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
462     bundle = os.path.join(out_dir, args.version)
463     out_prefix = args.out_uri.rstrip("/")
464     pushed = []
465     for fn in sorted(os.listdir(bundle)):
466         uri = f"{out_prefix}/weights/{args.version}/{fn}"
467         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
468         pushed.append(uri)
469     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
470     for u in pushed:
471         print("    ", u)
472     return 0
473
474 def build_parser() -> argparse.ArgumentParser:
475     p = argparse.ArgumentParser(prog="python -m backend.worker",
476                                 description="Storage-aware worker: pull ? run pipeline ?
477     push.")
478     p.add_argument("-v", "--verbose", action="store_true",
479                   help="DEBUG logging (the native detector command, frame counts, per-
480 stage detail)")
481     sub = p.add_subparsers(dest="cmd", required=True)
482
483     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
484     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
485 gcs://")
486     pi.add_argument("--out-uri", required=True, help="output prefix
487 (outputs/<video_id>/? is appended)")
488     pi.add_argument("--segmenter", default=None, help="default:
489 indexing.default_segmenter")
490     pi.add_argument("--config", default=None, help="config.json path (default:
491 ./config.json)")
492     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
493 dir)")
494     pi.add_argument("--max-frames", type=int, default=None)
495     pi.add_argument("--done-uri", default=None,
496                   help="M6: storage URI for a completion MARKER (video_id + rc)
497 written on a "
498                   "successful run, so a remote (Cloud Run) dispatcher's bucket-
499 poll terminates. "
500                   "Default-OFF: omit it for the normal local run.")
501     pi.add_argument("--set", action="append", default=[], metavar="k=v",
502                   help="config override, e.g. --set indexing.skip_ai_handoff=true")
503     pi.set_defaults(func=cmd_infer)
504
505     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
506 trajectories -> harness)")
507     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
508 videos/ for "
509                   "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
510     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
511 is appended)")

```

```

500         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
501         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
502                             help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
503                             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
504                             "(needs videos/ in the corpus; honours detector_impl).")
505         pt.add_argument("--segmenter", default="wasb_hybrid",
506                             help="trajectory hybrid whose detector is used for --trajectory-
source detector")
507         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
508         pt.add_argument("--config", default=None)
509         pt.add_argument("--scratch", default=None)
510         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
511         pt.add_argument("--frame-width", type=float, default=1280.0,
512                             help="synth trajectory frame width (detector probes the video)")
513         pt.add_argument("--set", action="append", default=[], metavar="k=v")
514         pt.set_defaults(func=cmd_train)
515         return p
516
517
518     def _setup_logging(verbose: bool) -> None:
519         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
520         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
521         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
522         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
523         entry), not at import, so importing the module never reconfigures a caller's
logging."""
524         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
525         # configured logging at import), which would leave the level at WARNING and re-
suppress the
526         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
527         logging.basicConfig(
528             level=logging.DEBUG if verbose else logging.INFO,
529             format="%(asctime)s [% (levelname)s] %(name)s: %(message)s",
530             datefmt="%H:%M:%S",
531             force=True,
532         )
533
534
535     def main(argv=None) -> int:
536         args = build_parser().parse_args(argv)
537         _setup_logging(getattr(args, "verbose", False))
538         return args.func(args)
539
540
541     if __name__ == "__main__":
542         sys.exit(main())
543

```

14. assistant (2026-06-22T14:44:40.643Z)

Now let me check `get\_compute\_backend` (the wiring that turns `compute\_target=cloud\_run` into a `CloudRunCompute`) in `backend/compute/\_\_init\_\_.py` and `base.py`.

15. user (2026-06-22T14:44:41.785Z)

```

1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5      ``None``
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
8      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
9
10     """
11
12     from __future__ import annotations
13
14     import os
15     from typing import Any, Optional
16
17     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20     __all__ = [
21         "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
22     ]
23
24     def _storage_dict(config: Any) -> dict:
25         storage = getattr(config, "storage", None)
26         if storage is None:
27             return {}
28         if hasattr(storage, "model_dump"):
29             return storage.model_dump() # type: ignore[no-any-return]
30         return dict(storage) if isinstance(storage, dict) else {}
31
32     def get_compute_backend(
33         config: Any,
34         *,
35         run_client: Optional[Any] = None,
36         policy: ExecutionPolicy = DEFAULT_POLICY,
37         token: Optional[str] = None,
38     ) -> Optional[ComputeBackend]:
39         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40         in-process (the default-OFF path).
41
42         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46         """
47         target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48         if target != "cloud_run":
49             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")

```

```

56     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57     client = run_client if run_client is not None else GoogleCloudRunJobClient()
58     return CloudRunCompute(
59         run_client=client, job_name=job_name, region=region, project=project,
60         storage_config=_storage_dict(config), policy=policy, token=token,
61     )
62

```

16. user (2026-06-22T14:44:42.157Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
5     DETECT?WINDOW?STITCH
6     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
7     (cloud-serving) ? without the core ever branching on the profile (D1).
8
9     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
10    non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
11    worker's
12    existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
13    inline").
14
15    """
16
17    from __future__ import annotations
18
19
20    from abc import ABC, abstractmethod
21    from dataclasses import dataclass, field
22    from typing import Optional
23
24
25    @dataclass(frozen=True)
26    class ComputeJobSpec:
27        """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
28        (re)compute
29        from the SAME input bytes, so the output always lands at
30        ``<out_uri>/outputs/<video_id>/`` ?
31        deterministic regardless of where the job ran."""
32
33        video_uri: str # input (gs://? for cloud; a path/local://
34        for local)
35        out_uri: str # output ROOT (gs://? bucket for cloud)
36        segmenter: Optional[str] = None
37        # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
38        frozen/hashable.
39        config_overrides: tuple[str, ...] = ()
40
41
42    @dataclass(frozen=True)
43    class ComputeResult:
44        """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
45        code
46        (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
47        timeout or an
48        empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
49        once the job's
50        done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
51        landed."""
52
53        returncode: int
54        outputs_uri: str = ""
55        video_id: str = ""
56        report: dict = field(default_factory=dict)
57

```

```

45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

17. assistant (2026-06-22T14:45:24.869Z)

I now have the complete picture. Let me verify the exact paths/marker key format by confirming the `_dispatch/<token>.done`` and reviewing `parse_uri`/`ref.name`` quickly for the marker filename used in the poll.

The map is complete. Here is the full picture.

How `compute_target=cloud_run` + `storage=gcs`` get set and flow to `get_compute_backend``

****1.** The `cloud-serving`` profile preset bundles all four knobs\*\* ? `presets.py:51-56``:
`python``

```

"cloud-serving": {
    "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
    "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
    "storage": {"backend": "gcs"},
},

```

The 1:1 profile?target map is `presets.py:32-36`` (`COMPUTE_TARGET_FOR_PROFILE["cloud-serving"] = "cloud_run``). Canonical enum tuples are `presets.py:27-28``.

****2.** The loader applies the preset only when explicitly selected** (`config.json`` deployment.profile` or `DEPLOYMENT_PROFILE`` env ? `loader.py:53-77`` `_resolve_explicit_profile``). On a hit it does a 3-way deep merge `defaults < preset < config.json`` (`loader.py:166-167``) and re-stamps the profile + back-fills the canonical `compute_target`` if config blanked it (`loader.py:170-187``). Auto-detect (`presets.py:109-132`` `suggest_profile``, keyed off `K_SERVICE``) only ever *suggests*, never applies (`loader.py:191-197``, D15). The typed mirror that pins parity is `DeploymentConfig`` ? `models.py:362,365`` (`compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"]``). Storage backend literal is `models.py:318``.

****3.** `cmd_infer`` consults the compute seam first\*\* ? `worker/___main__.py:168-170``:

```

python
compute = get_compute_backend(config)
if compute is not None:
    return _dispatch_remote(compute, args)

```

`get_compute_backend`` returns non-None **\*\*only\*\*** for `compute_target == "cloud_run`` (`compute/___init__.py:47-49``); every other target returns `None`` ? the inline in-process path runs (default-OFF seam). For cloud it lazy-imports and builds `CloudRunCompute``, pulling Cloud Run coordinates from env (`CLOUD_RUN_JOB`/`CLOUD_RUN_REGION`/`GOOGLE_CLOUD_PROJECT``), never config ? `compute/___init__.py:52-61``.

****4.** Dispatch + completion flow** (`compute/cloud_run.py``): `run_infer`` (`:103-132``) starts a Cloud Run **\*\*Job\*\*** execution running the *same* `backend.worker_infer`` argv, forced inline via `_DEFAULT_CONTAINER_OVERRIDES = ("deployment.compute_target=local_gpu", "indexing.detector_impl=native")`` (`:41-44``) so the container never re-dispatches, then ****bucket-polls**** the done-marker (`:119``, `poll_until`` + `_read_marker`` at `:135-140``). The container worker writes the marker at the end of `cmd_infer`` only when `--done-uri`` is passed (`worker/___main__.py:259-260``).

****Storage is fully credential-free + lazy****: ``gcs`` backend (``storage/gcs.py:17-25``) uses google ADC (``GOOGLE_APPLICATION_CREDENTIALS`/workload identity`), imported lazily only when a ``gs://`` URI is touched. ``acquire_local_input`` (``storage/__init__.py:59-74``) fetches a remote input to a scratch dir (local/bare = identity passthrough); the worker uses the lower-level ``storage.fetch`/`storage.put`` directly (``worker/__main__.py:179, 248``).

ON-BUCKET OUTPUT FORMAT a cloud ``worker infer`` writes (what the API ingests back)

Written to scratch ``outputs/<video_id>/`` (``worker/__main__.py:239-242``) then pushed to `**<out_uri>/outputs/<video_id>/<fn>`` for each file (``worker/__main__.py:244-249``). ``video_id`` = whole-file MD5 of the *\*pulled bytes\** (``worker/__main__.py:182-183``), so the path is deterministic regardless of where the job ran (``compute/base.py:22-24``).

File 1 ? ``outputs/<video_id>/intervals.csv``

Writer ``_write_intervals_csv`` ? ``worker/__main__.py:73-85``. Header + rows (``:78-84``):

```
| column | source | format |
|---|---|---|
| `start` | `s["start_time"]` | decimal seconds, `%.3f` |
| `end` | `s["end_time"]` | decimal seconds, `%.3f` |
| `shot_count` | `s["shot_count"]` | int or `""` |
| `ending_reason` | `s["ending_reason"]`, fallback `s["shot_count_confidence"]` | string or `""`
|
```

Semantics: decimal-second ``[start, end)`` rallies; docstring (``:74-75``) notes it is "the same schema as the rally-annotator labels" ? the join-friendly artifact.

File 2 ? ``outputs/<video_id>/report.json``

Writer ``_write_report_json`` ? ``worker/__main__.py:88-96``. Keys:

```
```json
{
 "video_id": "<md5>",
 "status": "success" | "failed",
 "segment_count": <int>,
 "segments": [{ "start": <float>, "end": <float>, ... }]
}
```
```

Note ``report.json`` segments carry **only** ``start`/`end`` (floats) ? ``shot_count`/`ending_reason`` live *only* in ``intervals.csv``. ``status`` is ``"success"`` when segmentation ran (``worker/__main__.py:214``), else ``"failed"`` (default ``:195``; set on validation/segmenter failure paths). The cloud dispatcher reads this report back into ``ComputeResult.report`` (``compute/cloud_run.py:129-132``).

File 3 ? the done/success marker: ``<out_uri>/_dispatch/<token>.done``

This is the success signal, written **separately** from the ``outputs/`` prefix. The dispatcher chooses the URI as ``f"{out_root}/_dispatch/{token}.done"`` (``compute/cloud_run.py:104-107``, ``token`` = injected or a fresh ``uuid4().hex`` ``:105``) and passes it as ``--done-uri``. The worker writes it via ``_write_done_marker`` ? ``worker/__main__.py:142-155`` ? **on success only** (``:259-260``). Marker JSON (``:147-148``):

```
```json
{"video_id": "<md5>", "returncode": <int>, "segment_count": <int>, "status": "<status>"}
```
```

Caveats the API must handle:

- **No failure marker.** A failed/hung container writes no marker; the dispatcher's bucket-poll exhausts its budget ? ``PolicyTimeout`` ? clean **rc 4** (``worker/__main__.py:132-139``, ``compute/base.py:38-43``). On the inline-success path the marker is always written with ``returncode=0`` hardcoded (``worker/__main__.py:260``).
- **Marker read defensiveness**: a missing/garbled ``returncode`` defaults to **failure (1)**, and a present-but-empty marker (no ``video_id``) is treated as failure (``compute/cloud_run.py:121-128``).

Ingestion contract summary (what the API needs)

1. Poll/read ``<out_uri>/_dispatch/<token>.done` ? get `video_id`, `returncode`, `status`, `segment_count`.`
2. On rc 0, fetch ``<out_uri>/outputs/<video_id>/report.json` (segment list + status) and/or `<out_uri>/outputs/<video_id>/intervals.csv` (the richer per-rally rows incl. `shot_count`/`ending_reason`).`
3. Key everything by ``video_id`` (whole-file MD5; the labels?video?DB join key, ``compute_video_id``).

Note: the worker writes **only** ``intervals.csv`` + ``report.json`` to the bucket (it lists scratch ``outputs/<video_id>/` and pushes each, `worker/__main__.py:244-249`). It does not push the SQLite DB, the highlight reel, or per-clip media ? the worker's DB (worker/__main__.py:189) and `emit_indexer_report` (:231-236) stay in the ephemeral scratch dir and are not uploaded. If the API needs the reel/DB, that is a gap (no compiled `highlights.mp4` is produced or pushed by `cmd_infer`).`

Relevant files (absolute):

- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\config\presets.py``
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\config\models.py``
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\config\loader.py``
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\__init__.py`` (the ``get_compute_backend`` seam)
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py``
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud_run.py``
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py`` (`cmd_infer` + the three writers ``_write_intervals_csv`` / ``_write_report_json`` / ``_write_done_marker``)
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\storage\__init__.py``,
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\storage\gcs.py``